Chaîne Podcast du Pi

Oct 21, 2026 · @Depret

En bref

Le Raspberry Pi fait la veille et la voix, Claude écrit le dialogue, et Google Drive sert de boîte aux lettres entre les deux. Le Google Sheet « Configuration podcasts » décide de tout : émissions, sources, jour, durée et voix. Rien n'est à modifier sur le Pi pour ajouter une émission ou une source.

Sources lues le 21 octobre 2026 : `README.md`, `BASCULE.md`, `Amélioration.txt`, `commun.py`, `veille.py`, `podcast.py`, `SKILL_podcast-ecriture.md` et `update_code.sh`, dans `Projet_Raspberry/Podcast`.

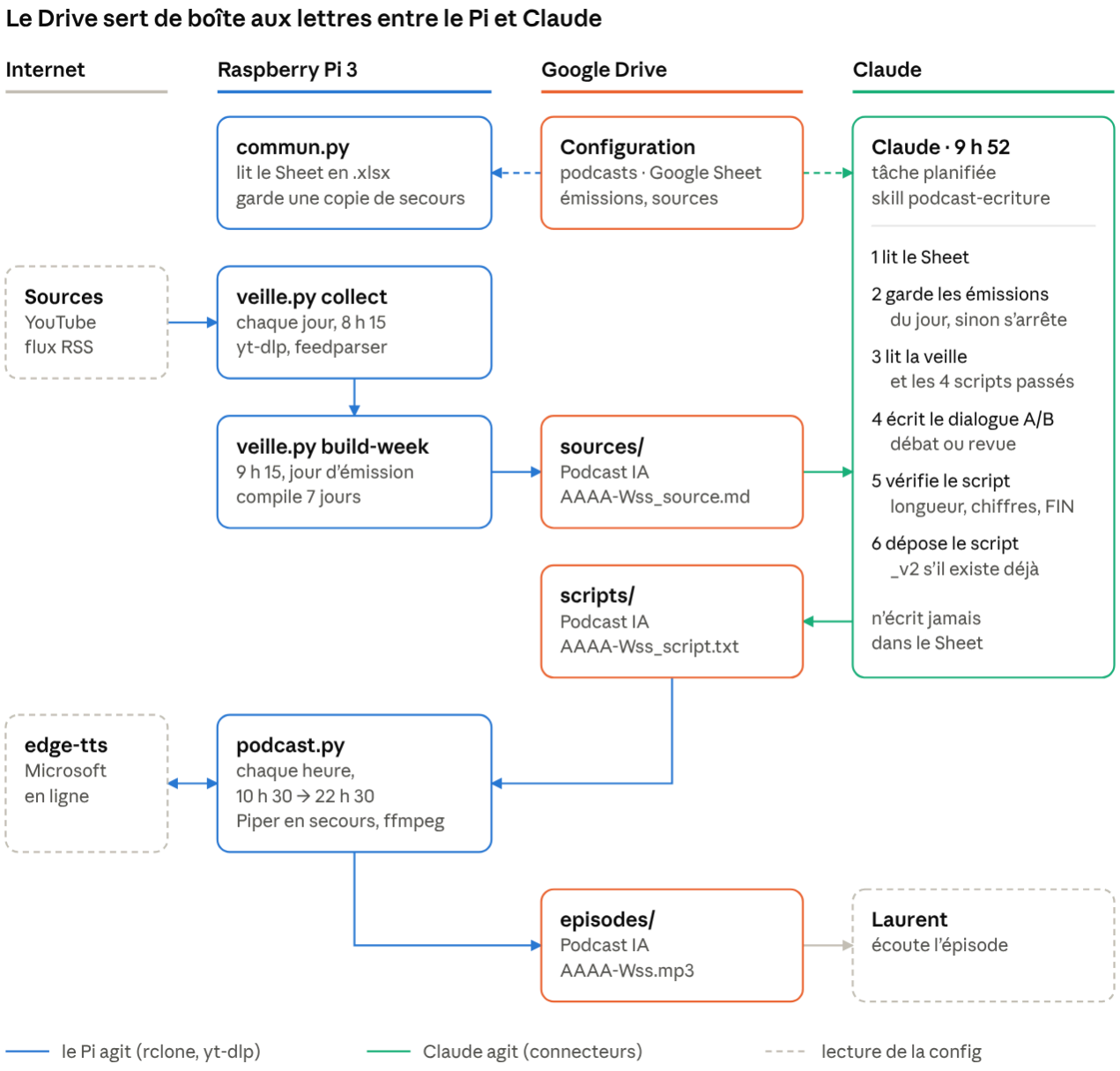
Une journée d'émission

Quatre passages se suivent chaque jour ; seuls les deux du milieu dépendent du jour de l'émission dans le Sheet.

1. **8 h 15 · Pi, chaque jour** : `veille.py collect` récupère les nouveautés de chaque source active.
2. **9 h 15 · Pi, le jour de l'émission** : `veille.py build-week` compile les 7 derniers jours et dépose la veille sur le Drive.
3. **9 h 52 · Claude, chaque jour** : la tâche planifiée lance le skill `podcast-ecriture`, qui dépose le script. Sans émission ce jour-là, il s'arrête.
4. **De 10 h 30 à 22 h 30, toutes les heures · Pi** : `podcast.py` transforme le nouveau script en MP3 et l'envoie sur le Drive.

Schéma des flux

Chaque dossier du Drive n'a qu'un écrivain : le Pi écrit `sources/` et `episodes/`, Claude écrit `scripts/`.



flux Pi, Drive, Claude · une semaine d'émission

Le Pi passe toujours par rclone (remote `podcast:`), Claude par ses connecteurs Google Drive et Google Sheets. Les deux ne se parlent jamais directement.

Rôles de chaque programme et agent

Programme ou agent	Où	Quand	Rôle	Lit	Écrit
Configuration podcasts (Google Sheet)	Drive	modifié à la main	Une ligne par émission (format, jour, durée, voix, prénoms, mots-clés, consignes) et un onglet de sources par émission (type, URL, langue, camp, thèse, filtre, active).	—	—
`commun.py`	Pi	à chaque lancement	Télécharge le Sheet en .xlsx et garde la dernière copie si le Drive ne répond pas. Fournit rclone avec délai et 3 essais, le journal et le choix des émissions du jour.	Sheet (.xlsx)	`configuration.xlsx`, journaux
`veille.py collect`	Pi, ~/veille-env	chaque jour, 8 h 15	Récupère les nouveautés : flux RSS (feedparser, BeautifulSoup) et vidéos YouTube avec sous-titres (yt-dlp). Filtre par mots-clés, écarte le déjà-vu.	YouTube, RSS, `seen.json`	`daily/AAAA-MM-JJ.jsonl`
`veille.py build-week`	Pi	9 h 15, jour d'émission	Compile 7 jours en Markdown : sources avec camp et thèse, puis contenus. L'envoie lui-même sur le Drive.	`daily/`	`sources/AAAA-Wss_source.md`
Tâche planifiée « Podcast IA – script du dimanche »	Claude	chaque jour, 9 h 52	Déclenche le skill ; s'arrête sans rien écrire les jours sans émission.	—	—

Programme ou agent	Où	Quand	Rôle	Lit	Écrit
Skill `podcast-ecriture`	Claude	lancé par la tâche ou à la demande	Écrit le dialogue A / B (Camille et Thomas pour Podcast IA), en débat ou revue, en langue parlée. N'invente aucun chiffre, vérifie le script avant dépôt. Sauvegarde : `SKILL_podcast-ecriture.md`.	Sheet (connecteur), `sources/`, 4 derniers scripts	`scripts/AAAA-Wss_script.txt`
`podcast.py`	Pi, ~/piper-env	chaque heure, 10 h 30 → 22 h 30, verrou `flock`	Prend la version de script la plus récente, fait lire les répliques par edge-tts (Piper en secours), assemble, encode en MP3 avec ffmpeg et l'envoie. Reprend là où il s'était arrêté.	`scripts/`, `state.json`	`episodes/AAAA-Wss.mp3`
edge-tts · Piper	Internet · Pi	pendant `podcast.py`	edge-tts : voix neuronales Microsoft en ligne. Piper : voix locales fr_FR-siwis (A) et fr_FR-tom (B), plus lentes.	texte	WAV
`update_code.sh`	Pi	chaque matin, par cron	Récupère le code sur GitHub, teste la compilation, remplace les trois scripts dans `~/veille`. Garde l'ancienne version si le test échoue.	GitHub `lolo_python`	`~/veille/*.py`

Programme ou agent	Où	Quand	Rôle	Lit	Écrit
cron	Pi	en permanence	Lance chaque programme à son heure, met à jour yt-dlp le dimanche à 10 h 45. Partagé avec la synchro Garmin (remote `gdrive:`).	—	`veille.log`, `podcast.log`

Rangement des fichiers

Sur le Drive, chaque sous-dossier n'a qu'un seul écrivain : le Pi pour `sources/` et `episodes/`, Claude pour `scripts/`.

Google Drive · dossier Podcasts (racine du remote rclone `podcast:`)

```
Podcasts/
├── Configuration podcasts      (Google Sheet)
├── Podcast IA/
│   ├── sources/    ← Pi     · AAAA-Wss_source.md
│   ├── scripts/    ← Claude · AAAA-Wss_script.txt
│   └── episodes/   ← Pi     · AAAA-Wss.mp3
└── Podcast Crypto/             (créé le 21/10, vide)
```

Raspberry Pi · dossier personnel

```
veille/          commun.py, veille.py, podcast.py
veille-env/      feedparser, bs4, yt-dlp, openpyxl
piper-env/       piper-tts, edge-tts, openpyxl
piper-voices/    siwis (A), tom (B)
podcast-data/
├── configuration.xlsx      copie de secours du Sheet
├── veille.log · podcast.log
└── ia/
    ├── daily/ · weekly/ · seen.json
    ├── scripts/ · work/
    └── episodes/state.json
```

Mise à jour du code

Le code va du PC au Pi en passant par GitHub ; le skill, lui, s'enregistre à part dans Claude.

1. **PC** : `lolo_python/Projet_Raspberry/Podcast`, où le code est écrit et corrigé.
2. **GitHub** : dépôt `lolo_python`, branche `main`. Le Pi en garde une copie en lecture seule.
3. **Pi** · **`update_code.sh`**, chaque matin : récupère, teste avec `py_compile`, copie dans `~/veille` si le test passe. Les données, les environnements Python et rclone ne sont jamais touchés.
4. **Claude** : le skill `podcast-ecriture` s'enregistre dans Claude ; `SKILL_podcast-ecriture.md` n'en est que la sauvegarde.

Points de vigilance

Un problème reste à corriger dans le code ; les autres sont des vérifications sur le Pi ou des détails.

- [ ] **À corriger · repli sur Piper trop rapide** : un seul morceau raté 3 fois chez edge-tts, soit environ 30 secondes sans Wi-Fi, et tout l'épisode passe sur Piper. La synthèse prend alors des heures sur le Pi 3.
- [ ] **À vérifier · deux lancements en même temps** : la crontab de `BASCULE.md` lance `podcast.py` derrière `flock -n /tmp/podcast.lock`. Vérifier que la ligne est en place.
- [ ] **À vérifier · state.json** : `~/podcast-data/ia/episodes/state.json` doit exister, sinon `podcast.py` refait les MP3 de toutes les semaines passées.
- [ ] **Mineur · veille** : une vidéo sans date n'est jamais écartée et peut arriver sans texte ; les sources désactivées apparaissent avec « 0 éléments ».
- [ ] **Mineur · skill** : la sauvegarde `SKILL_podcast-ecriture.md` n'a pas encore la règle « script plus récent que la source ».